Tutorial 5: DISTINCT Pitfalls, NULL Pitfalls, and Outer Joins

CSCI3170 Introduction to Database Systems

Learning Objectives

In this Side Activity, you will learn:

- How **DISTINCT** works on whole rows, on multiple columns, and inside aggregate functions.
- Why **NULL** behaves differently from ordinary values, including three-valued logic and aggregate behavior.
- How to use `IS NULL`, `NVL`, and `COALESCE` correctly.
- How **LEFT**, **RIGHT**, and **FULL OUTER JOIN** preserve unmatched rows and introduce NULL values.
- How **DISTINCT**, **NULL**, and **outer joins** interact in subtle but important ways.

Part 0: Getting Started

We continue with the same **University schema**. If your tables are still in place, run a quick sanity check:

```
SELECT COUNT(*) AS num_students   FROM student;
SELECT COUNT(*) AS num_courses    FROM course;
SELECT COUNT(*) AS num_sections   FROM section;
SELECT COUNT(*) AS num_takes_rows FROM takes;
```

Expected: 13 students, 13 courses, 15 sections, 22 takes rows. If your numbers differ, re-run:

```
@DDL.sql
@smallRelationsInsertFile.sql
```

See Side Activities from Side Activities 9 and 10 for extended context

Two ideas from the previous Side Activities matter a lot today:

- In Side Activity 9, we saw `COUNT(DISTINCT ...)` and learned that **DISTINCT** removes duplicates before aggregation.
- In Side Activity 10, we saw that `NOT IN` can be dangerous when a subquery result contains **NULL**.

Today we look much more closely at **DISTINCT**, **NULL**, and the **outer joins** that naturally create NULL-filled rows.

Part 1: DISTINCT in More Detail

Key Concept: DISTINCT removes duplicate rows, not duplicate columns

`DISTINCT` is applied to the **entire row produced by the SELECT list**. If you select one column, duplicates are removed based on that column. If you select two columns, duplicates are removed based on the **pair**.

Demo 1: DISTINCT on multiple columns

Compare a one-column `DISTINCT` with a two-column `DISTINCT`:

```
SELECT DISTINCT semester
FROM section
ORDER BY semester;

SELECT DISTINCT c.dept_name, s.semester
FROM section s
JOIN course c ON s.course_id = c.course_id
ORDER BY c.dept_name, s.semester;
```

The first query asks: "Which semester names appear at all?" The second asks: "Which *(department, semester)* combinations appear?" Because `DISTINCT` is applied to the full row, the second query keeps one row for each unique pair.

Demo 2: How DISTINCT treats NULL

`DISTINCT` treats all NULL values as one duplicate group:

```
SELECT DISTINCT grade
FROM takes
ORDER BY grade;
```

In our data, there is one row where `grade` is NULL. Even if there were many such rows, `SELECT DISTINCT grade` would show only **one** NULL in the output.

DISTINCT and WHERE do not treat NULL the same way

`DISTINCT` groups all NULLs together for duplicate elimination, but comparisons such as `NULL = NULL` in a `WHERE` clause are **UNKNOWN**, not TRUE. This difference surprises many beginners.

Demo 3: DISTINCT vs GROUP BY

Sometimes `DISTINCT` and `GROUP BY` can produce the same visible result:

```
SELECT DISTINCT dept_name
FROM student
ORDER BY dept_name;

SELECT dept_name
FROM student
GROUP BY dept_name
ORDER BY dept_name;
```

These return the same department list. As a rule of thumb:

- Use `DISTINCT` when you simply want unique rows.
- Use `GROUP BY` when you want one row per group *and* also want aggregate values such as `COUNT`, `AVG`, or `SUM`.

Demo 4: DISTINCT inside an aggregate

`DISTINCT` can also appear *inside* an aggregate function:

```
SELECT SUM(c.credits) AS sum_all_credits,
       SUM(DISTINCT c.credits) AS sum_distinct_credit_values
FROM takes t
JOIN course c ON t.course_id = c.course_id
WHERE t.ID = '12345';
```

For student `12345`, the joined rows have credit values `4, 4, 3, 3`. Therefore:

- `SUM(c.credits)` adds all four values and returns `14`.
- `SUM(DISTINCT c.credits)` first removes duplicate *credit values*, leaving only `4` and `3`, and returns `7`.

Important subtlety

`SUM(DISTINCT c.credits)` removes duplicates of the **expression value** `c.credits`, not duplicates of the course. Two different courses with the same credit value still collapse into one value inside the aggregate.

Part 2: NULL in More Detail

Key Concept: Three-valued logic

With NULL, a Boolean condition may evaluate to **TRUE**, **FALSE**, or **UNKNOWN**. In SQL, rows are kept only when the `WHERE` condition is TRUE.
Examples:

- `NULL = NULL` is UNKNOWN.
- `NULL <> 5` is UNKNOWN.
- `NULL IS NULL` is TRUE.

Demo 5: `= NULL` vs `IS NULL`

The classic beginner mistake:

```
SELECT ID, course_id, semester, year, grade
FROM takes
WHERE grade = NULL;

SELECT ID, course_id, semester, year, grade
FROM takes
WHERE grade IS NULL;
```

The first query returns **zero rows**. The second correctly returns Tanaka's `BIO-301` row, because `IS NULL` is the proper NULL test.

Demo 6: NULL propagation

NULL often propagates through expressions:

```
SELECT ID, salary, salary + NULL AS salary_plus_null
FROM instructor
WHERE ID = '22222';

SELECT ID, name, name || NULL AS name_concat_null
FROM instructor
WHERE ID = '22222';
```

In Oracle, arithmetic with NULL returns NULL, but string concatenation with NULL behaves like concatenation with an empty string.

Demo 7: Aggregates and NULL

Aggregate functions do *not* all treat NULL the same way:

```
SELECT COUNT(*) AS total_rows,
       COUNT(grade) AS non_null_grades
FROM takes;
```

In our data:

- `COUNT(*)` counts all 22 rows.
- `COUNT(grade)` skips the one NULL grade, so it returns 21.

`AVG`, `SUM`, `MIN`, and `MAX` also ignore NULL values rather than treating them as zero.

Demo 8: `NOT IN` + NULL trap

First, a `NOT IN` that works correctly — find students who have no advisor:

```
SELECT s.ID, s.name
FROM student s
WHERE s.ID NOT IN (
  SELECT a.s_ID FROM advisor a
)
ORDER BY s.ID;
```

This returns 4 students: Brandt, Williams, Sanchez, Snow. Now watch what happens if a single NULL sneaks into the subquery result:

```
SELECT s.ID, s.name
FROM student s
WHERE s.ID NOT IN (
  SELECT a.s_ID FROM advisor a
  UNION ALL
  SELECT NULL FROM dual
)
ORDER BY s.ID;
```

The result is **zero rows**. Adding one NULL to the set causes `NOT IN` to evaluate to UNKNOWN for every outer row, silently discarding all results. This is a deliberately constructed counterexample, but in real queries the NULL often comes from a nullable column in the subquery rather than from `dual`.

The NULL-safe alternative:

```
SELECT s.ID, s.name
FROM student s
WHERE NOT EXISTS (
  SELECT 1
  FROM advisor a
  WHERE a.s_ID = s.ID
)
ORDER BY s.ID;
```

This correctly returns the same 4 students regardless of NULLs.

Rule of thumb

When you mean "find rows with no match", prefer `NOT EXISTS`. It is NULL-safe. Use `NOT IN` only when you are certain the subquery cannot return NULL.

Demo 9: NVL and COALESCE

Replace NULL with a display value:

```
SELECT ID, course_id,
       NVL(grade, 'N/A') AS grade_display,
       COALESCE(grade, 'N/A') AS grade_display_std
FROM takes
ORDER BY ID, course_id, year, semester;
```

`NVL` is Oracle-specific and takes two arguments. `COALESCE` is standard SQL and can take two or more expressions, returning the first non-NULL one.

Demo 10: ORDER BY and NULL

Oracle lets you control where NULL values appear in the sort order:

```
SELECT ID, course_id, grade
FROM takes
WHERE course_id IN ('BIO-101', 'BIO-301')
ORDER BY grade;

SELECT ID, course_id, grade
FROM takes
WHERE course_id IN ('BIO-101', 'BIO-301')
ORDER BY grade NULLS FIRST;
```

By default, Oracle places NULLs **last** in ascending order. `NULLS FIRST` and `NULLS LAST` let you control this explicitly.

Part 3: Outer Joins

Key Concept: INNER vs LEFT / RIGHT / FULL OUTER JOIN

- **INNER JOIN**: keep only matching rows.
- **LEFT JOIN**: keep all rows from the left table; fill unmatched right-side columns with NULL.
- **RIGHT JOIN**: keep all rows from the right table; fill unmatched left-side columns with NULL.
- **FULL OUTER JOIN**: keep all rows from both sides; unmatched columns on either side become NULL.

Outer joins are the most natural way to create NULL-filled rows during a join.

Demo 11: LEFT JOIN — all students and their advisors

Compare an inner join with a left join:

```
SELECT s.ID, s.name AS student_name, i.name AS advisor_name
FROM student s
JOIN advisor a ON s.ID = a.s_ID
JOIN instructor i ON a.i_ID = i.ID
ORDER BY s.ID;

SELECT s.ID, s.name AS student_name, i.name AS advisor_name
FROM student s
LEFT JOIN advisor a ON s.ID = a.s_ID
LEFT JOIN instructor i ON a.i_ID = i.ID
ORDER BY s.ID;
```

The inner join returns only students who have an advisor. The left join returns **all 13 students**; for students without an advisor, `advisor_name` is NULL.

Demo 12: LEFT JOIN + IS NULL as an anti-join

Once a left join introduces NULLs on unmatched rows, you can filter on those NULLs:

```
SELECT s.ID, s.name
FROM student s
LEFT JOIN advisor a ON s.ID = a.s_ID
WHERE a.i_ID IS NULL
ORDER BY s.ID;
```

This finds students with **no advisor**. It is the left-join version of an anti-join and is closely related to the `NOT EXISTS` pattern from Side Activity 10.

Demo 13: LEFT JOIN + COUNT(*) vs COUNT(column)

This is one of the most important outer-join traps:

```
SELECT s.ID, s.name,
       COUNT(*) AS count_star,
       COUNT(t.course_id) AS count_course_id
```

```
FROM student s
LEFT JOIN takes t ON s.ID = t.ID
GROUP BY s.ID, s.name
ORDER BY s.ID;
```

For students who have taken courses, the two counts are the same. But for Snow, who has never taken any course:

- `COUNT(*)` returns `1`, because the left join still produces one output row.
- `COUNT(t.course_id)` returns `0`, because the unmatched `t.course_id` is NULL and NULL is skipped by `COUNT(column)`.

Outer join counting rule

After an outer join, use `COUNT(non_null_column)` when you want the number of actual matches. Using `COUNT(*)` often overcounts unmatched rows.

Demo 14: RIGHT JOIN and FULL OUTER JOIN

RIGHT JOIN is just the mirror image of LEFT JOIN:

```
SELECT a.s_ID, s.name AS student_name, i.name AS advisor_name
FROM student s
RIGHT JOIN advisor a ON s.ID = a.s_ID
LEFT JOIN instructor i ON a.i_ID = i.ID
ORDER BY a.s_ID;
```

This query keeps all rows from `advisor` even if the left side were missing. In our data, every `advisor.s_ID` has a matching student, so it behaves like an inner join.

FULL OUTER JOIN keeps unmatched rows from both sides:

```
SELECT COALESCE(s.name, i.name) AS person_name,
       s.ID AS student_id,
       i.ID AS instructor_id
FROM student s
FULL OUTER JOIN instructor i
  ON s.name = i.name
ORDER BY person_name;
```

This query lets you see who appears only as a student, only as an instructor, or on both sides. In our data, `Brandt` appears on both sides, while most names appear on only one side.

Part 4: Self Exercises

All questions use the **University schema**.

Q1. (DISTINCT on multiple columns) List all distinct `(semester, year)` pairs that appear in `section`. Output: `semester, year`. Order by `year`, then `semester`.

Q2. (COUNT(*) vs COUNT(column)) Write one query that shows `COUNT(*)` and `COUNT(grade)` for the `takes` table. Briefly explain why the two numbers are different.

Q3. (IS NULL) Find all rows in `takes` whose grade is still NULL. Output: `ID, course_id, semester, year`. Order by `ID`.

Q4. (NVL / COALESCE) Display all rows in `takes`, but replace a NULL grade with `'N/A'`. Output: `ID, course_id, semester, year, grade_display`. Order by `ID, course_id, year, semester`.

Q5. (LEFT JOIN) List every student together with the name of their advisor. Students without an advisor must still appear. Output: `student_id, student_name, advisor_name`. Order by `student_id`.

Q6. (LEFT JOIN + IS NULL) Find all students who do not have an advisor. Use `LEFT JOIN` together with `IS NULL`. Output: `ID, name`. Order by `ID`.

Q7. (LEFT JOIN + COUNT + DISTINCT) For every student, show how many **distinct courses** they have taken. Students who have never taken any course must appear with `0`. Use `COUNT(DISTINCT ...)` to avoid counting the same course twice when a student retakes it. Output: `ID, name, num_courses`. Order by `ID`.

Q8. (Challenge: DISTINCT + NULL + outer-style reasoning) Find all departments for which the number of distinct non-NULL grades earned in that department's courses is strictly greater than 3. Output: `dept_name, num_distinct_grades`. Order by `dept_name`.

Hint: Join `department` to `course`, then to `takes`. `COUNT(DISTINCT grade)` automatically ignores NULL grades.

End of Tutorial 5.